

EXAME ÉPOCA NORMAL (2ª PARTE, COM CONSULTA, DURAÇÃO: 75 min., 9 valores)

Nota: Nas perguntas seguintes, quando solicitado, deve usar troços de código Java essencial, podendo intermediar com linhas de comentário. Escrever código irrelevante com definições de *import* de *packages*, *static main*, *System.out.println*; *public* etc. é perda de tempo que lhe pode faltar para aspetos mais importantes.

9. [3 val.] Existe uma empresa de jogos online que tem, dispersos pela cidade, imensos quiosques onde os jogadores vão presencialmente jogar. O jogo consiste em os jogadores tentarem acertar numa sequência de 6 números inteiros entre 1 e 99 gerada aleatoriamente no servidor em cada jogo. Existe um prémio em euros que é atribuído ao primeiro jogador que acertar na sequência dos 6 números. Assim, cada quiosque tem múltiplos terminais de jogo, onde se executa uma aplicação cliente que acede a um dos múltiplos servidores existentes na sede da empresa de jogos. O cliente e o servidor interagem por gRPC com o seguinte contrato:

<pre>service CalcService { rpc jogada(stream Aposta) returns (stream Resultado); }</pre>	
<pre>message Aposta { int32 NIF = 1; // Número Identificação Fiscal string IBAN = 2; // conta bancária do jogador repeated int32 seq = 3; // 6 números da aposta }</pre>	<pre>message Resultado { int32 NIF = 1; boolean acertou = 2; optional float premio = 3; }</pre>

- a) [1.5 val.] Apresente o código Java essencial da classe que implementa o contrato no servidor, sem ser necessário indicar o código do método *static main* de inicialização do servidor. Assuma que existe no servidor uma variável *static* com os 6 números aleatórios que permitem atribuir o prémio. O valor do prémio será depositado na conta bancária do jogador vencedor, definida no campo IBAN da aposta (*Coloque no código um comentário a indicar a ação do depósito do prémio e onde é debitado o custo de cada jogada*);
- b) [1.5 val.] Apresente o código Java essencial da aplicação cliente que permite enviar uma sequência de apostas e mostrar na consola os resultados de cada aposta.
10. [3 val.] Diariamente em cada concelho de Portugal (total de 308) existem sensores de temperatura que produzem leituras da temperatura ambiente nas 24 horas do dia (uma leitura por cada hora). No Instituto Português do Mar e Atmosfera (IPMA) existe um sistema suportado no RabbitMQ que tem um *Exchange* do tipo FANOUT para cada concelho (308 *Exchanges* de nome EX1, ... ,EX308) que recebem as mensagens dos sensores (temperatura, hora). Cada *Exchange* tem associada uma *queue* (308 *queues* de nome Q1, ... ,Q308) em que cada *queue* tem associado uma aplicação consumidora das mensagens recebidas para as armazenar numa base de dados (*Coloque no código um comentário a indicar a ação de update da base de dados*).
- No entanto, verificou-se posteriormente que existiam pessoas interessadas em receber as mensagens de um único concelho em determinadas condições.
- Apresente o código Java essencial de uma aplicação que recebe as mensagens de um único concelho e mostra na consola as mensagens com temperaturas acima dos 35 graus centígrados.
11. [3 val.] [Não precisa apresentar código] Considere um cenário de uma organização que tem um cluster formado por várias máquinas virtuais (VM) com os endereços (IP1, IP2, ..., IPN) onde se executam múltiplos *containers* Docker. Alguns desses *containers* estão num estado de erro, mas a ocupar recursos das VM, necessitando que periodicamente se aceda VM a VM para visualizar o estado e eventualmente remover *containers*.
- Utilizando um ou mais dos *middleware* estudados**, proponha uma possível solução através de um diagrama de componentes e respetiva descrição das interações e respetivas API, para que:
- Seja possível remotamente obter o estado dos *containers* em todas as VM;
 - Seja possível remover *containers* em estado de erro, libertando assim recursos nas múltiplas VM.